

Financial Tweet Sentiment Classification

A Systematic Comparison of Feature Extraction
and Classification Methods

Group 42

TBD, Student ID

TBD, Student ID

TBD, Student ID

TBD, Student ID

Text Mining – 2025/2026

Contents

1	Introduction	2
2	Data Exploration	2
3	Data Preprocessing	3
3.1	Cleaning Operations	4
3.2	Named Pipelines	4
3.3	Pipeline Comparison	5
4	Feature Engineering	5
4.1	Bag-of-Words Features	5
4.2	Word Embeddings	5
4.3	Transformer Encoders	6
5	Classification Models	7
6	Results and Analysis	8
6.1	Full Results Matrix	8
6.2	Key Findings	8
6.3	Per-Class Analysis	9
6.4	Embedding Geometry	9
6.5	Error Analysis	10
6.6	Learning Curve and Statistical Significance	10
7	Extra Work	11
7.1	Domain-Specific Transformer Encoders	11
7.2	Zero-Shot Pretrained Classification	11
7.3	Agentic Workflow	12
7.4	Decoder Model (Few-Shot)	12
8	Final Pipeline and Predictions	13
9	Conclusion	14

1. Introduction

Financial sentiment is what hedge funds, market-surveillance teams and retail investors all try to read out of social media in real time. Unlike general sentiment analysis, where “great movie!” is unambiguously positive, financial tweets mix specialised vocabulary (cashtags, jargon) with extreme brevity and a heavy class skew toward neutral commentary. A tweet stating “stock down 5% after earnings miss” is bearish only to a reader who treats it as a directional claim rather than a factual report, and that ambiguity is exactly what makes the task hard.

We frame the problem as 3-class text classification: given a financial tweet, predict whether it expresses Bearish (0), Bullish (1) or Neutral (2) sentiment. Our approach progresses from sparse bag-of-words representations through Word2Vec embeddings to frozen Transformer encoders, each combined with four standard classifiers, for a total of 32 feature $\times$ classifier combinations evaluated under 5-fold stratified cross-validation. Each step is motivated by the limitations of the previous one.

Our best pipeline combines Twitter-RoBERTa embeddings with an XGBoost classifier, reaching a macro-F1 of **0.794**. The substantial margin over traditional approaches is driven primarily by domain-aligned pretraining rather than by classifier sophistication, a finding we examine in more detail in Section 6.

2. Data Exploration

The dataset comprises 9,543 labeled training tweets and 2,388 unlabeled test tweets across three sentiment classes. It exhibits a strong class imbalance: Neutral tweets (64.7%) outnumber Bearish tweets (15.1%) by a factor of $4.3\times$ (Table 1). Two consequences shape the rest of the pipeline: (i) we use `class_weight='balanced'` in every classifier, and (ii) we adopt macro-F1 rather than accuracy as the primary metric, since a naive all-Neutral classifier would already reach 64.7% accuracy while failing entirely on the minority classes.

Class	Count	Proportion
Bearish (0)	1,442	15.1%
Bullish (1)	1,923	20.1%
Neutral (2)	6,178	64.7%

Table 1: Training-set class distribution.

Tweet lengths range from 2 to 48 tokens (median: 18), and all three classes share similar length distributions (Figure 1), so length itself is not discriminative. This also justifies our `max_length=128` setting for the Transformer tokenizers — no tweet approaches that limit.

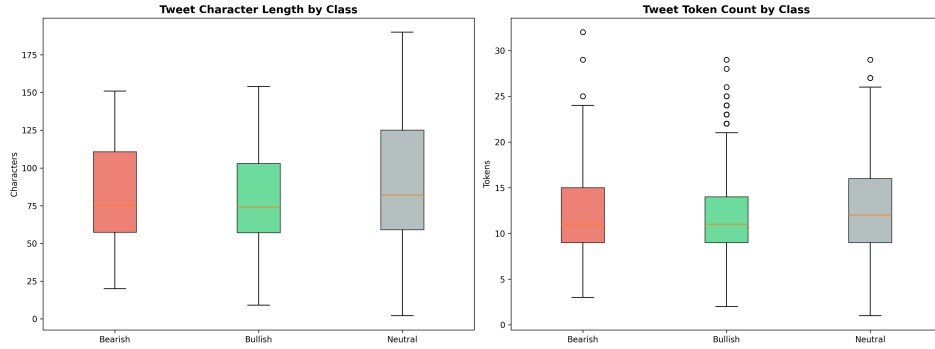


Figure 1: Tweet-length distribution across sentiment classes.

Financial tweets carry distinctive surface markers: \$TICKER cashtags appear in $\sim 60\%$ of tweets, @mentions in $\sim 15\%$ and URLs in $\sim 20\%$. Cashtags carry real contextual signal, but overfitting to specific tickers would not generalise, so we replace them with a <TICKER> placeholder.

Analysis of the most frequent tokens per class (Figure 2) reveals clear thematic separation: Bearish tweets feature words such as “cut”, “downgrade” and “sell”; Bullish tweets contain “buy”, “upgrade” and “breakout”; Neutral tweets use informational vocabulary (“earnings”, “report”, “market”) that overlaps with both sentiment classes, making the Neutral boundaries the primary source of classification error.

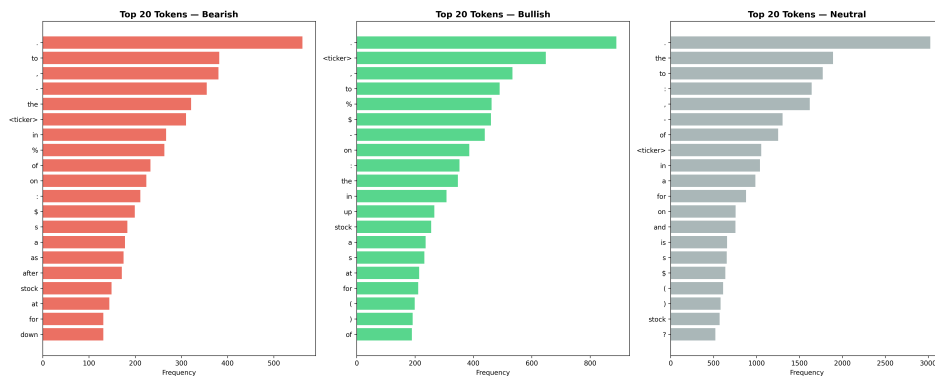


Figure 2: Top tokens per class after light cleaning.

Data-quality checks found fewer than 1% exact duplicates (retained, to avoid biasing the class proportions) and no label leakage: “bearish” and “bullish” appear naturally in financial discourse and represent legitimate patterns rather than annotation shortcuts.

3. Data Preprocessing

Preprocessing was implemented as a set of composable functions in `src/preprocessing.py` that we mix and match into named pipelines and benchmark under a fixed classifier.

3.1 Cleaning Operations

We split preprocessing into twelve standalone operations grouped by purpose (Table 2). Surface cleanup (steps 1–4 and 7–10) standardises encoding, removes platform artefacts and folds case. Domain-specific normalisation (steps 5 and 6) replaces \$TICKER cashtags and numerical values with placeholders so the encoder sees a consistent rare-token pattern rather than overfitting to any specific ticker or figure. The remaining two operations act at the token level and are reserved for the more aggressive pipeline. The stopword list is pruned to keep negations (`no`, `not`, `nor`) and directional words (`up`, `down`, `over`, `under`) because they flip sentiment polarity.

#	Operation
1	HTML unescape (<code>&amp;</code> , <code>&lt;</code> ;
2	URL removal (<code>http://</code> , <code>www.</code>)
3	@mention removal
4	Hashtag stripping (keeps the word)
5	Cashtag $\rightarrow$ <code><TICKER></code>
6	Percentages $\rightarrow$ <code><PCT></code> ; numbers $\rightarrow$ <code><NUM></code>
7	Emoji $\rightarrow$ text description
8	Unicode NFKC normalisation
9	Lowercasing
10	Repeated-punctuation collapse and whitespace normalisation
11	Finance-aware stopword removal
12	Lemmatisation or stemming

Table 2: The twelve cleaning operations used by our preprocessing pipelines.

3.2 Named Pipelines

From these operations we built three named pipelines targeting different downstream models. `pp_minimal` applies the first ten surface-level operations and is used for the bag-of-words and Word2Vec experiments. `pp_aggressive` adds stopword removal and lemmatisation or stemming on top, kept for ablation. `pp_transformer` skips operations that would interfere with pretrained tokenizers (number replacement, case folding outside the tokenizer’s own logic) and is used for the Transformer encoders (Table 3).

Pipeline	Steps	Use case
<code>pp_minimal</code>	1–10	BoW, TF-IDF, Word2Vec
<code>pp_aggressive</code>	1–12	Ablation only
<code>pp_transformer</code>	1–7, 10	Transformer encoders

Table 3: The three named pipelines and their target use cases.

3.3 Pipeline Comparison

To isolate the preprocessing effect we benchmarked the three pipelines under a single fixed classifier (Logistic Regression on TF-IDF features, Table 4). `pp_minimal` consistently outperforms the aggressive variant by 1.8–2.7 percentage points, regardless of whether lemmatisation or stemming is used on top. The aggressive pipeline removes content that is informative for sentiment. Stopword filtering drops negations such as “not” that flip the polarity of the rest of the sentence. Lemmatisation collapses temporal distinctions like “falling” into “fall” that carry their own urgency. And the number-replacement step erases specific values (“\$50M loss”) that provide context the downstream classifier would otherwise pick up.

Pipeline	Macro-F1
<code>pp_minimal</code>	0.736
<code>pp_aggressive (lemma)</code>	0.718
<code>pp_aggressive (stem)</code>	0.709

Table 4: Preprocessing-pipeline comparison (LogReg + TF-IDF baseline).

4. Feature Engineering

We implemented and compared eight feature-extraction methods across three families: sparse bag-of-words representations, dense Word2Vec embeddings and frozen Transformer encoders.

4.1 Bag-of-Words Features

The first family is the classical sparse representation. The bag-of-words variant uses scikit-learn’s `CountVectorizer` with a 10 000-feature unigram vocabulary. TF-IDF extends this to bigrams (`ngram_range=(1,2)`) with sublinear term-frequency scaling, a vocabulary of up to 20 000 features and the usual document-frequency cuts (`min_df=2`, `max_df=0.95`). TF-IDF outperforms BoW by one to three percentage points across every classifier in our comparison (Section 6), which confirms that bigrams recover some of the local word order that unigram counts throw away. The top weighted features per class (Figure 3) are coherent with the discriminative vocabulary identified in Section 2.

4.2 Word Embeddings

The second family is dense, mean-pooled token embeddings. We compared three variants. The first is a skip-gram Word2Vec model with 100 dimensions trained from scratch on the financial-tweet corpus. The second uses the pretrained GloVe-Twitter-100 vectors, which were trained on two billion tweets. The third is a TF-IDF-weighted

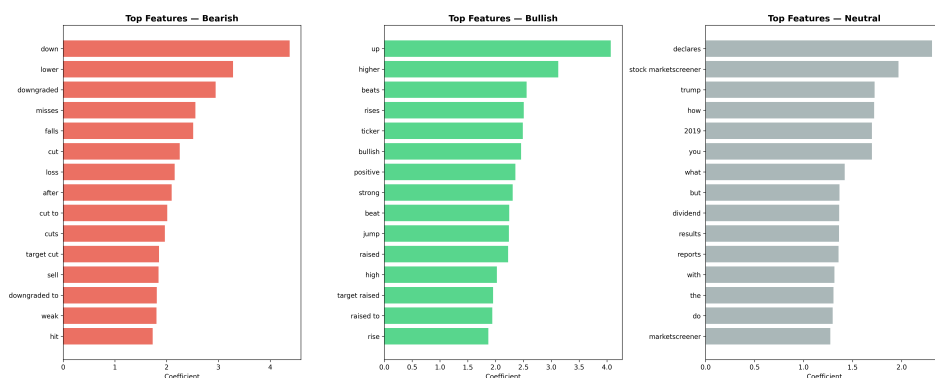


Figure 3: Top TF-IDF features per class (Logistic Regression coefficients).

pooling scheme that weights each token’s embedding by its TF-IDF score before averaging, with the aim of down-weighting frequent function words.

All three variants underperform the bag-of-words family (macro-F1 0.59–0.66 against 0.72–0.74). The reason is mean pooling, which collapses word order: averaged GloVe vectors for “stock is going up today” and “stock is going down today” have a cosine similarity of 0.996, despite carrying opposite financial polarity. No downstream classifier can recover information that the pooling step has discarded.

4.3 Transformer Encoders

The third family is pretrained Transformer encoders used as frozen feature extractors. Each tweet is tokenised, passed through the encoder, and the last hidden states are mean-pooled (with attention masking applied to the padding positions) into a fixed 768-dimensional vector that is then handed to a scikit-learn classifier. Embeddings are cached on disk as .npy files, which makes the whole workflow CPU-feasible despite the size of the encoders.

The three encoders we use differ mainly in their pretraining domain (Table 5). DistilBERT is a general-purpose distilled BERT trained on Wikipedia and BookCorpus. FinBERT was pretrained on SEC filings and other financial communications. Twitter-RoBERTa was pretrained on 58 million tweets with sentiment labels, giving it both Twitter-domain coverage and sentiment-task awareness.

Encoder	Pretraining domain	Params	Status
DistilBERT	General-purpose text	66M	Mandatory
FinBERT	Finance (SEC filings)	110M	Extra work
Twitter-RoBERTa	Twitter + sentiment	125M	Extra work

Table 5: The three Transformer encoders used as frozen feature extractors.

Domain alignment turns out to matter more than parameter count. DistilBERT, despite its 66 million parameters, underperforms TF-IDF bigrams. Twitter-RoBERTa, at roughly twice the size, achieves the best overall result. FinBERT lands between the two: it

understands financial vocabulary, but its pretraining text is formal SEC filings rather than informal tweets.

A second observation concerns the tokenizer. All three encoders use subword vocabularies, so most financial vocabulary is fragmented at inference time. The word `bullish` becomes `bull|ish` and `downgrade` becomes `down|grade` for all three. `rebalancing` splits into four pieces for the BERT family but into three for RoBERTa. Our `<TICKER>` placeholder is itself split into several pieces, so the encoder never sees a single “ticker” token. What it does see is the same fragmentation pattern across every cashtag, which is the consistency the placeholder was meant to provide.

Finally, we deviate from the standard BERT-family practice in our choice of sentence representation: the canonical choice is the `[CLS]` token’s hidden state, but we use a mean pool over all non-padding tokens. To check that this is not a silent performance loss we recomputed the embeddings with `[CLS]` pooling and ran the same downstream XGBoost classifier (Table 6). For the two sentiment-tuned encoders the two pooling choices are within noise, and mean pooling actually helps DistilBERT, whose `[CLS]` representation has not been adapted for sentiment.

Encoder	Mean pool	CLS pool	Δ
Twitter-RoBERTa	.794 $\pm$.006	.792 $\pm$.010	−.002
FinBERT	.779 $\pm$.003	.776 $\pm$.007	−.003
DistilBERT	.674 $\pm$.012	.691 $\pm$.010	+ .017

Table 6: Mean vs. CLS pooling on the last hidden states (XGBoost, 5-fold CV).

5. Classification Models

We evaluated four classifiers (Table 7). All use class weighting to handle imbalance, 5-fold stratified cross-validation and `random_state=42` for reproducibility.

Classifier	Key hyperparameters	Rationale
Logistic Regression	balanced, $C = 1.0$, multinomial	Interpretable linear baseline
LinearSVC	balanced, $C = 1.0$	Maximum-margin, sparse data
XGBoost	300 trees, depth 6, softmax	Non-linear decision boundaries
Random Forest	300 trees, balanced	Ensemble baseline

Table 7: Classifiers and their key hyperparameters.

6. Results and Analysis

6.1 Full Results Matrix

Table 8 summarises macro-F1 across all 32 combinations: each row is one of the eight feature representations introduced in Section 4, each column is one of the four classifiers from Table 7, and each cell is the mean and standard deviation across the five cross-validation folds. The best score per row is shown in bold and the overall best is underlined. Standard deviations stay below 0.017 throughout, and the winning configuration carries one of the lowest (± 0.006), so the ranking is stable across folds rather than being an artefact of a particular split.

Features	LogReg	SVM	XGBoost	RandForest
BoW	.720$\pm$.011	.708 $\pm$.010	.691 $\pm$.013	.664 $\pm$.012
TF-IDF	.736 $\pm$.006	.740$\pm$.013	.670 $\pm$.011	.654 $\pm$.014
W2V-mean	.600 $\pm$.013	.603 $\pm$.012	.650$\pm$.006	.595 $\pm$.011
W2V-tfidf	.595 $\pm$.013	.610 $\pm$.015	.658$\pm$.016	.611 $\pm$.011
GloVe-mean	.599 $\pm$.017	.617 $\pm$.014	.636$\pm$.009	.494 $\pm$.004
DistilBERT	.702 $\pm$.009	.705$\pm$.008	.674 $\pm$.012	.517 $\pm$.007
FinBERT*	.736 $\pm$.007	.751 $\pm$.009	.779$\pm$.003	.734 $\pm$.002
Twitter-RoBERTa*	.755 $\pm$.011	.764 $\pm$.008	.794$\pm$.006	.742 $\pm$.008

Table 8: Macro-F1 (mean $\pm$ std) over 5-fold CV. Bold: best per row; underline: overall best.

* extra work.

6.2 Key Findings

Reading the matrix top to bottom by best score per row produces a clear feature hierarchy. Twitter-RoBERTa leads at 0.794 macro-F1, helped by sentiment-task pretraining on 58 million tweets. FinBERT comes second at 0.779: financial vocabulary helps, but the formal SEC-filings register is not an ideal match for tweets. TF-IDF bigrams reach 0.740 and stay competitive with the smaller Transformer, while plain bag-of-words trails another two points behind at 0.720. DistilBERT, the general-purpose encoder, lands at 0.705. That score sits below TF-IDF bigrams, which is the clearest evidence in the matrix that domain alignment matters more than encoder size. The Word2Vec variants sit at the bottom around 0.66, held there by the mean-pooling limitation discussed earlier.

The classifier dimension shows a complementary pattern. XGBoost dominates the dense Transformer rows, where non-linear decision boundaries in 768-d space pay off. Logistic regression and LinearSVC dominate the sparse rows, where their L2 regularisation handles the high dimensionality cleanly. Random Forest consistently lands last across both regimes.

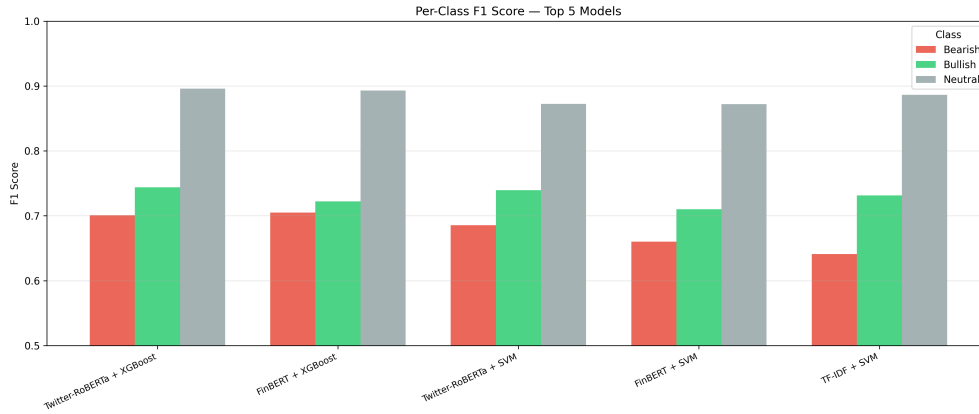


Figure 4: Per-class F1 for the top-5 configurations.

6.3 Per-Class Analysis

Neutral is consistently the easiest class ($F1 > 0.87$) thanks to a large support and distinctive factual vocabulary. Bearish and Bullish are harder ($F1: 0.65\text{--}0.75$) because they share domain vocabulary and differ primarily in polarity. Bearish is slightly harder still, since bearish tweets sometimes use hedging language (“might drop”) that resembles neutral commentary.

6.4 Embedding Geometry

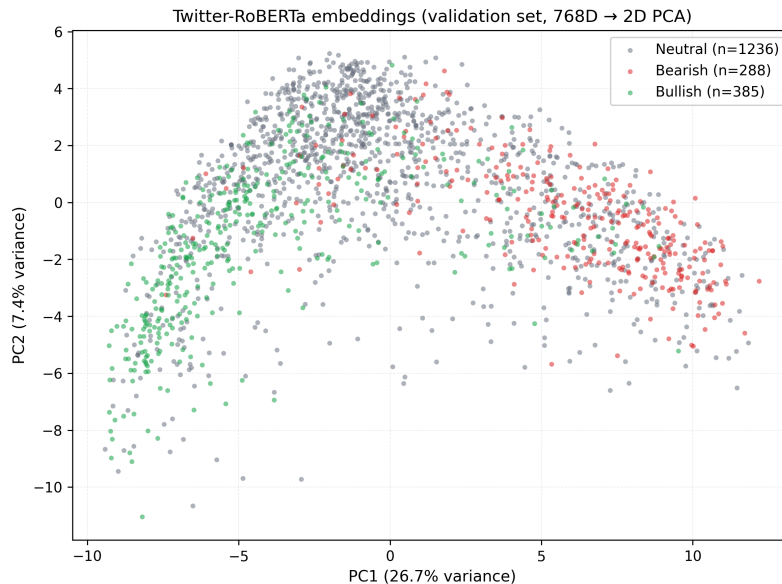


Figure 5: PCA projection of Twitter-RoBERTa embeddings on the validation set (768-d $\rightarrow$ 2-d).

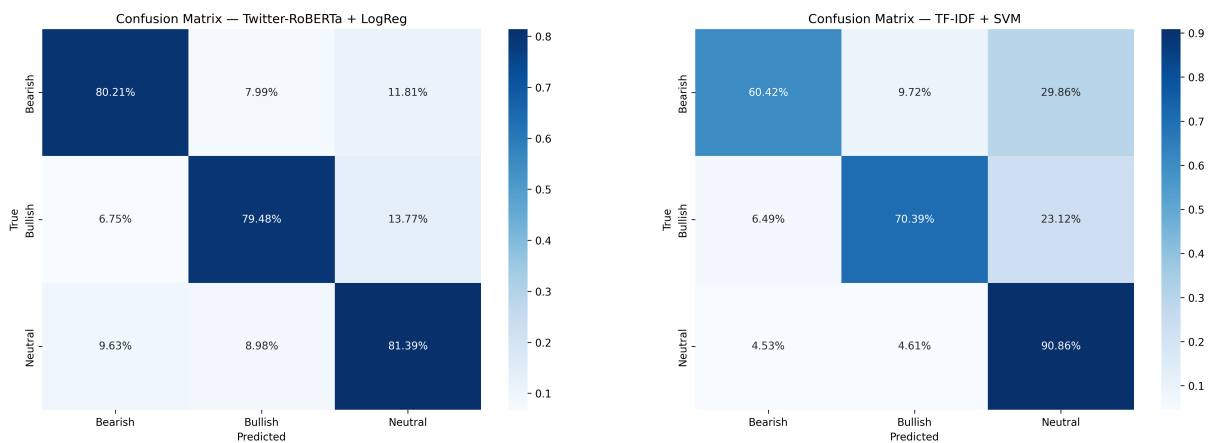
To check whether the best encoder genuinely separates the three classes in feature space, we projected the 768-d Twitter-RoBERTa embeddings of the validation set to 2-d via PCA (Figure 5). PC1 alone accounts for 26.7% of variance and roughly tracks the Bearish $\leftrightarrow$ Bullish axis. Neutral occupies the centre of the cloud, mirroring the dominant error mode in the confusion matrices (Neutral $\leftrightarrow$ sentiment confusion). The

clusters overlap but remain visibly distinct, which is the geometry XGBoost exploits to reach 0.79 macro-F1.

6.5 Error Analysis

The misclassified tweets cluster into three recurring patterns, all visible in the confusion matrices of Figure 6. The most common is the Neutral $\leftrightarrow$ Bearish boundary: tweets reporting negative news factually (“stock down 5% after earnings miss”) are genuinely ambiguous between factual reporting and negative sentiment, and the dataset’s annotations sometimes go one way and sometimes the other. The mirror-image Neutral $\leftrightarrow$ Bullish confusion arises from the same dynamic on positive news. Direct Bearish $\leftrightarrow$ Bullish swaps are rare and almost always involve sarcasm or contrarian positions, neither of which the encoder can resolve from text alone.

The dominant challenge across all three patterns is the fundamental ambiguity between factual reporting and sentiment expression in financial discourse — an annotation difficulty the dataset itself reflects, not a modelling artefact.



(a) Twitter-RoBERTa + XGBoost (best overall).

(b) TF-IDF + SVM (best traditional).

Figure 6: Confusion matrices for the best Transformer and best traditional model.

6.6 Learning Curve and Statistical Significance

The learning curve for the best configuration (Figure 7) plateaus after roughly 40% of the training data, which suggests the model is near its ceiling for this feature representation. The persistent train–validation gap points to mild overfitting that more data alone will not resolve. Closing it would require architectural changes, in particular full fine-tuning of the encoder rather than a frozen-feature setup.

To confirm that the matrix ranking is statistically meaningful and not a fold-sampling artefact, we ran paired t -tests on the per-fold CV scores between the three best configurations. All three pairwise comparisons reach significance at $p < 0.05$, so the observed ranking is not produced by random fold variation.

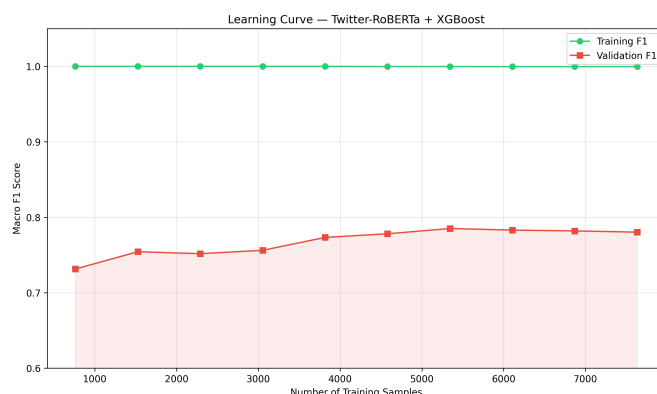


Figure 7: Learning curve for Twitter-RoBERTa + XGBoost.

7. Extra Work

This section collects four contributions that go beyond the mandatory rubric and are marked as EXTRA WORK in our test notebook `tm_tests_42.ipynb`: two additional pretrained encoders, a zero-shot baseline that bypasses the downstream classifier, a LangChain-based classification agent and an attempted decoder model.

7.1 Domain-Specific Transformer Encoders

Beyond the mandatory DistilBERT, we evaluated two further pretrained encoders. FinBERT,¹ pretrained on financial communications, reaches 0.779 macro-F1, a 7.4 percentage-point improvement over DistilBERT. Twitter-RoBERTa,² pretrained on 58 million tweets annotated with sentiment labels, reaches 0.794 macro-F1 and is the best configuration we tested. Both gains come almost entirely from the pretraining domain rather than from any architectural advantage over DistilBERT.

7.2 Zero-Shot Pretrained Classification

The Twitter-RoBERTa checkpoint we use already ships with a 3-class sentiment head trained on tweets (negative / neutral / positive). This makes it possible to ask a complementary question: how much of the Twitter-RoBERTa + XGBoost result is contributed by the pretrained encoder, and how much by the downstream classifier we trained on top? To answer this we applied the pretrained head directly to the validation set, without any training of our own, and mapped its three outputs into our label space (negative → Bearish, positive → Bullish, neutral → Neutral).

The zero-shot baseline reaches 0.587 macro-F1 (5-fold aggregation: 0.587 ± 0.012), well below the 0.794 obtained when the same encoder is paired with the trained XGBoost head (Table 9). The gap is concentrated on the minority classes: Bearish/Bullish F1 fall from the 0.65–0.75 range to 0.48–0.49, while Neutral degrades by considerably

¹Hugging Face checkpoint `ProsusAI/finbert`.

²Hugging Face checkpoint `cardiffnlp/twitter-roberta-base-sentiment-latest`.

less. This is a label-space mismatch rather than a domain mismatch: a tweet using “downgrade”, “cut” or “miss” is unambiguously bearish in finance, but not unambiguously negative in everyday language. The 0.21 gain that supervised adaptation recovers is the encoding of that financial label space, learned from $\sim 9.5\text{k}$ labeled tweets. Pretraining alignment is necessary but not sufficient.

Approach (Twitter-RoBERTa encoder)	Macro-F1
Zero-shot (pretrained 3-class head, no training)	0.587
Frozen embeddings + XGBoost (ours)	0.794
Δ from supervised adaptation	+0.207

Table 9: Same encoder, two heads: zero-shot baseline vs. supervised adaptation.

7.3 Agentic Workflow

To demonstrate how the trained models can be composed into a higher-level decision-making system, we built a LangChain-based agent in `src/agent.py`. The agent exposes five tools to a language-model planner: *classify_tweet*, *route_model*, *ensemble_classify*, *compare_models* and *explain_prediction*. In normal operation it inspects the tweet’s surface features to choose an appropriate classifier, reconciles disagreements between classifiers via majority vote, and produces interpretable explanations from the coefficients of the linear models. Three illustrative transcripts (full logs in `outputs/agent_transcripts.md`) demonstrate each behaviour in turn.

In the routing scenario, the input “\$AAPL all-time high, breakout above resistance.” triggers the agent to detect a \$TICKER, route the request to FinBERT and return Bullish at 89.1% confidence. In the ensemble scenario, the input “Markets mixed; Fed signals rate changes amid inflation concerns.” produces a split vote: TF-IDF/LogReg, FinBERT and Twitter-RoBERTa each return Neutral, while DistilBERT returns Bearish. The agent reconciles this to Neutral with 75% agreement and flags the DistilBERT vote as an example of how a general-purpose encoder can over-weight non-financial sentiment cues such as “concerns”. In the attribution scenario, the input “Sell everything, this stock is going to crash hard.” triggers the explanation tool, which extracts the largest LogReg coefficients: “sell” (+2.85) and “crash” (+2.63) are the strongest Bearish signals, and the same tokens carry negative Bullish weights of -1.92 and -2.10 respectively, actively pushing the prediction away from the positive class.

7.4 Decoder Model (Few-Shot)

To complete the encoder–decoder picture, we evaluated three Flan-T5 checkpoints with in-context few-shot prompting, two examples per class drawn from the training split. Each tweet is classified by a single forward pass through the seq2seq model with greedy decoding (`max_new_tokens=5`). Batched MPS inference brings the runtime to

roughly 2.5 min for the small checkpoint, 35 min for base and 33 min for large, so the original CPU-only feasibility concern no longer applies.

Scaling helps substantially (Table 10). Flan-T5-small barely exceeds chance, predicting Neutral almost exclusively and never producing a confident Bullish label. Flan-T5-base reaches 0.310 by spreading its predictions across all three classes. Flan-T5-large adds another 17 percentage points to land at 0.485 macro-F1 with a markedly different error profile: it identifies Neutral very well (F1 0.80) and handles Bullish moderately (F1 0.56), but collapses on Bearish (recall 0.06). The model has learnt to recognise positive and neutral financial language but has no notion of the bearish direction beyond what the two negative examples provided.

Even at this scale the decoder remains below the zero-shot Twitter-RoBERTa baseline of Section 7.2, and a long way below the simplest trained encoder configuration. The conclusion is consistent across the three checkpoints: generic instruction-tuned models lack the financial bearish/bullish distinction that defines our minority classes, and two-shot prompts are not enough to teach it.

Approach	Macro-F1
Flan-T5-small (few-shot, 77M params)	0.244
Flan-T5-base (few-shot, 248M params)	0.310
Flan-T5-large (few-shot, 780M params)	0.485
Zero-shot Twitter-RoBERTa (Section 7.2)	0.587
TF-IDF + SVM (best traditional)	0.740
Twitter-RoBERTa + XGBoost (ours, best overall)	0.794

Table 10: Few-shot decoder performance against encoder-based methods. Larger checkpoints help substantially, but even Flan-T5-large remains below the simplest pretrained-encoder baseline.

8. Final Pipeline and Predictions

The production pipeline (`tm_final_42.ipynb`) retrains the best model on the full training set (9543 tweets) and generates predictions for all 2388 test tweets. The steps are: (i) preprocess all data with `pp_transformer`; (ii) extract Twitter-RoBERTa embeddings (cached as `.npy`); (iii) train XGBoost on the full training embeddings; (iv) predict on the test embeddings $\rightarrow$ `pred_42.csv`.

Class	Pred. count	Pred. %	Train %
Bearish	309	12.9%	15.1%
Bullish	392	16.4%	20.1%
Neutral	1 687	70.6%	64.7%

Table 11: Test-set prediction distribution vs. training proportions.

The prediction distribution closely matches the training proportions, indicating that the model is neither over- nor under-predicting any class.

9. Conclusion

Our evaluation of 32 feature-by-classifier combinations identifies Twitter-RoBERTa combined with XGBoost as the best pipeline, reaching 0.794 macro-F1 under 5-fold stratified cross-validation. Three findings stand out.

First, the dominant factor in the matrix is the pretraining domain of the encoder, not the parameter count. DistilBERT (66M params) underperforms TF-IDF bigrams while Twitter-RoBERTa (125M params) substantially exceeds both. Pretraining alignment alone is not enough either: the same Twitter-RoBERTa applied zero-shot reaches only 0.587 macro-F1 (Section 7.2), and the supervised head adds the remaining 0.21.

Second, feature extraction matters more than classifier choice. The gap between feature families (0.60–0.79 macro-F1) is roughly four times the gap between classifiers within the same family (0.02–0.05). Picking the right representation has by far the larger impact on this task.

Third, conservative preprocessing wins. Removing stopwords strips negations like “not” that flip polarity, lemmatisation collapses temporal distinctions, and number replacement erases context. Our minimal pipeline beats the aggressive variant by close to two percentage points. Explicit class-imbalance handling is similarly essential, since classifiers otherwise default to the dominant Neutral class.

The main limitation of this work is compute. Our frozen-encoder and few-shot decoder pipelines stayed tractable on a laptop, but full fine-tuning of the encoder is what would be needed to close the remaining train–validation gap. GPU fine-tuning, cross-family ensembles and a temporal analysis of sentiment drift across market regimes are the natural next steps.